

INSTITUTO TECNOLÓGICO DE MATEHUALA

CARRERA: Ingeniería en Sistemas Computacionales | SEMESTRE: Quinto semestre

ASIGNATURA: Fundamentos de Ingeniería de Software (Unidad 3)

BITÁCORA GENERAL DE PROYECTO

SISTEMA DE E-COMMERCE: GroStop (ECommerce-Grocery-Store)

CÉLULA DE DESARROLLO: Alpha Logic Systems (Equipo 3)

UBICACIÓN Y FECHA: Matehuala, San Luis Potosí — Octubre de 2026

INTEGRANTES:

- Oscar Hernández Banda — Product Owner / Líder de Requisitos
 - Yordi Hernández Coronado — Arquitecto de Software / Modelado UML
 - Kenia Guadalupe Hernández Torres — Desarrollador Backend / Full Stack / Core
 - Cristofer Yahir Herrera Rodriguez — Control de Versiones (Git) / Tester
 - Ana Fernanda Loera Mendoza — Analista de Datos / Persistencia
-

ÍNDICE DE CONTENIDO

1. Introducción
2. Objetivo
3. Contexto del Proyecto
4. Integrantes y Asignación de Roles
5. Plan de Trabajo Original (Cronograma de 15 Días)
6. Actividades Planificadas vs. Actividades Realizadas
7. Bitácora Cronológica General
8. Problemas Técnicos Reales, Causas, Soluciones y Resultados
9. Registro de Comandos Utilizados
10. Pruebas del Sistema
11. Arquitectura y Funcionamiento Técnico
12. Trabajo Colaborativo y Coordinación
13. Resultados Obtenidos
14. Conclusión Grupal
15. Evidencias

1. INTRODUCCIÓN

El presente documento constituye la Bitácora General Grupal de desarrollo del proyecto "ECommerce-Grocery-Store", denominado comercialmente GroStop, elaborado por la célula de trabajo Alpha Logic Systems (Equipo 3) en el marco de la asignatura Fundamentos de Ingeniería de Software (Unidad 3) de la carrera de Ingeniería en Sistemas Computacionales del Tecnológico Nacional de México, sede Matehuala.

El desarrollo de software profesional no solo implica la codificación de nuevas funcionalidades desde cero, sino también la capacidad de abordar codebases existentes, analizar sus dependencias, corregir errores de flujo e incompatibilidades de entorno, y estructurar modelos de datos persistentes. En este reporte se formaliza el proceso de recepción, estabilización, depuración de backend, migración de repositorio y análisis de arquitectura del sistema GroStop.

2. OBJETIVO

Documentar de manera rigurosa, transparente y cronológica las actividades planificadas y ejecutadas por la célula Alpha Logic Systems durante la depuración, configuración, modelado e implementación técnica del proyecto GroStop, detallando los problemas de compatibilidad y lógica encontrados, las soluciones aplicadas, las pruebas respaldadas y los resultados obtenidos sin recurrir a suposiciones o datos no verificados.

3. CONTEXTO DEL PROYECTO

El proyecto ECommerce-Grocery-Store (GroStop) es una plataforma web orientada al comercio electrónico de productos de abarrotes. La célula Alpha Logic Systems asumió el proyecto con el objetivo de realizar la exploración funcional, el despliegue local, la estabilización del entorno ejecutable en Python, el análisis de la persistencia de datos relacional y el modelado UML formal.

El código base original presentaba incompatibilidades críticas de librerías vinculadas a versiones recientes del intérprete de Python, errores en la lógica de cálculo de carritos y cupones de descuento, así como duplicidad de registros en la base de datos MySQL/MariaDB, los cuales fueron abordados progresivamente.

4. INTEGRANTES Y ASIGNACIÓN DE ROLES

- **Oscar Hernández Banda (Product Owner / Líder de Requisitos):** Levantamiento y formalización de requisitos, análisis de flujos de usuario del sistema (Cliente/Administrador) e interacción/enlace directo con el docente para el seguimiento de entregables.
- **Yordi Hernández Coronado (Arquitecto de Software / Modelado UML):** Definición y análisis de la arquitectura del sistema, modelado de Diagramas de Casos de Uso, Diagramas de Clases y Diagramas de Entidades.

- **Kenia Guadalupe Hernández Torres (Desarrollador Backend / Full Stack / Core):** Desarrollo y estabilización del backend en Flask (Python), integración con la base de datos MySQL, configuración del entorno de desarrollo ejecutable y solución de errores del núcleo del sistema.
- **Cristofer Yahir Herrera (Control de Versiones (Git) / Tester):** Gestión y centralización del control de versiones en Git y GitHub, administración del flujo de trabajo colaborativo y aplicación de pruebas respaldadas por el equipo.
- **Ana Fernanda Loera Mendoza (Analista de Datos / Persistencia):** Análisis del esquema de base de datos MySQL (grostop.sql), análisis de tablas y sus relaciones relacionales, validación de la persistencia de datos y análisis de permisos e integridad de almacenamiento.

5. PLAN DE TRABAJO ORIGINAL (CRONOGRAMA DE 15 DÍAS)

- **Días 1-2: Recepción y Exploración:** Transferencia e inspección del código fuente base, reconocimiento del proyecto y asignación formal de roles dentro de la célula.
- **Días 3-5: Despliegue Local:** Instalación de dependencias, configuración de entornos virtuales, ejecución local del servidor web y resolución de errores iniciales de arranque.
- **Días 6-8: Exploración Funcional:** Navegación del sistema desplegado, auditoría de módulos activos y hallazgos técnicos en las rutas y flujos del sistema.
- **Días 9-11: Diagramación y Documentación:** Elaboración de modelos UML, documentación de arquitectura, casos de uso y análisis estructural de la base de datos.
- **Día 12: Checkpoint de Avance:** Evaluación del estado del proyecto e inspección intermedia de entregables ante el docente.
- **Días 13-14: Desarrollo de Funcionalidad:** Corrección de errores en la lógica de negocio, ajuste de endpoints de Flask y refactorización de scripts de base de datos.
- **Día 15: Presentación Final:** Cierre del periodo de trabajo y entrega final de la documentación y sistema funcionando.

6. ACTIVIDADES PLANIFICADAS VS. ACTIVIDADES REALIZADAS

6.1 Actividades Planificadas

- Recepción completa del código fuente original.
- Instalación y ejecución sin contratiempos mediante el entorno estándar.
- Mapeo de casos de uso completos del sistema de e-commerce.
- Generación de diagramas de clases y entidad-relación detallados.
- Implementación de nuevas funcionalidades de negocio durante los Días 13-14.
- Ejecución del Checkpoint de avance en el Día 12 y la Presentación Final en el Día 15.

6.2 Actividades Realizadas

- **Migración y Centralización:** Se migró el proyecto y se centralizó el control de versiones en el repositorio de GitHub <https://github.com/aakkadn-beep/ECommerce-Grocery-Store.git>.
- **Estabilización de Entorno Python:** Se identificó que la ejecución se realizaba con Python global, generando conflictos de dependencias. Se resolvió creando y activando un entorno virtual dedicado con `python -m venv venv` y `source venv/Scripts/activate`, registrando las dependencias en `requirements.txt`.
- **Depuración de Backend (Flask):** Se logró la estabilización del backend en Flask, corrigiendo de manera efectiva la lógica del flujo de checkout, la gestión de compras y el cálculo de cupones.
- **Auditoría y Corrección de Base de Datos (MySQL/MariaDB):** Se revisó el script `grostop.sql` validando sus 13 tablas relacionales, la persistencia de datos y corrigiendo errores específicos en `Percentage_Discount` y en la inserción de registros duplicados (`ON DUPLICATE KEY UPDATE`).
- **Resolución de Incompatibilidades de Librerías:** Se corrigieron fallas de compilación de PyYAML, falta del módulo `pyparsing` e incompatibilidades de Flask-MySQLdb.

7. BITÁCORA CRONOLÓGICA GENERAL

Periodo	Actividad	Responsable	Problema	Solución	Resultado
Días 1-2	Recepción, exploración del código fuente y centralización en GitHub.	Célula Alpha Logic Systems / Cristofer Yahir Herrera	Dispersión inicial del código fuente original.	Migración y centralización del código en el repositorio GitHub.	Repositorio GitHub configurado y sincronizado para el equipo.
Días 3-5	Despliegue local, creación de entorno virtual y	Kenia Guadalupe Hernández Torres /	Ejecución en Python global provocaba errores de compilación	Recreación del entorno con <code>python -m venv venv</code> , activación con <code>source</code>	Entorno ejecutable con Python estabilizado y archivo <code>requirements.txt</code>

	resolución de dependencias.	Cristofer Yahir Herrera	(PyYAML) y librerías faltantes (pyparsing).	venv/Scripts/activate, e instalación fija con pip install --only-binary :all: -r requirements.txt.	generado.
Días 6-8	Exploración funcional, ejecución del servidor con python run.py e inspección de rutas.	Kenia Guadalupe Hernández Torres / Oscar Hernández Banda	Fallo en la importación al arrancar el servidor en market/routes.py por falta de pyparsing.	Instalación explícita de la librería pyparsing dentro del entorno virtual.	Ejecución del comando python run.py sin errores de importación en http://127.0.0.1:5000/ .
Días 9-11	Diagramación UML y análisis de persistencia del archivo grostop.sql.	Yordi Hernández Coronado / Ana Fernanda Loera Mendoza	Conflicto en la tabla al insertar registros duplicados e incoherencia en Percentage_Discount.	Implementación de ON DUPLICATE KEY UPDATE y corrección del tipo/lógica del porcentaje de descuento.	Esquema de 13 tablas en MySQL/MariaDB validado con persistencia correcta.
Día 12	Checkpoint de Avance.	Oscar Hernández Banda	Información no proporcionada por el equipo.	Información no proporcionada por el equipo.	Información no proporcionada por el equipo.
Días 13-14	Desarrollo de Funcionalidad y estabilización de flujos de checkout y cupones.	Kenia Guadalupe Hernández Torres / Ana Fernanda Loera Mendoza	Incompatibilidad de Flask-MySQLdb==1.0.1 e imprecisiones en el cálculo de compras.	Actualización a Flask-MySQLdb==2.0.0 (usando PyMySQL como alternativa) y refactorización del flujo en Flask.	Flujo de checkout, procesamiento de compras y lógica de cupones estabilizados.
Día 15	Presentación Final.	Oscar Hernández Banda / Célula Alpha Logic Systems	Estado de la presentación final: Información no proporcionada por el equipo.	Información no proporcionada por el equipo.	El cronograma contempla esta fase; la realización del evento queda sin registro expreso.

8. PROBLEMAS TÉCNICOS REALES, CAUSAS, SOLUCIONES Y RESULTADOS

Problema 1: Conflicto en inserción de registros duplicados

- **Problema:** Conflicto en la inserción de registros duplicados en la base de datos MySQL/MariaDB.
- **Causa:** Intentos de inserción de llaves o registros existentes durante transacciones o carga de datos iniciales sin contemplar la actualización de los mismos.
- **Solución:** Inclusión de la sentencia ON DUPLICATE KEY UPDATE en las consultas de inserción para gestionar la duplicidad.
- **Resultado:** Prevención de interrupciones por violaciones de clave primaria o única durante la ejecución e inserción de datos.

Problema 2: Incompatibilidad del esquema de descuentos

- **Problema:** Incompatibilidad e imprecisión en el esquema de descuentos dentro de la base de datos y el backend.
- **Causa:** Definición o tratamiento inadecuado del campo Percentage_Discount al aplicar los cálculos matemáticos del carrito de compras.
- **Solución:** Corrección realizada en el modelo y en las consultas para procesar adecuadamente la columna Percentage_Discount.
- **Resultado:** Cálculo correcto y preciso del monto total del carrito al aplicar cupones o promociones con descuento porcentual.

Problema 3: Dependencia faltante pyparsing

- **Problema:** Dependencia faltante de la librería pyparsing al ejecutar la aplicación.
- **Causa:** Al ejecutar la aplicación con python run.py, se lanzó la excepción de sistema: ModuleNotFoundError: No module named 'pyparsing', generada desde el módulo market/routes.py.
- **Solución:** Instalación manual y explícita del paquete pyparsing dentro del entorno virtual activo.
- **Resultado:** Resolución del error de importación en market/routes.py, permitiendo la correcta inicialización de las rutas de Flask.

Problema 4: Error de compilación de PyYAML

- **Problema:** Error de compilación e instalación de la librería PyYAML.
- **Causa:** Incompatibilidad directa del instalador de código fuente de PyYAML con la versión de Python, arrojando errores de compilación Cython.
- **Solución:** Forzado de instalación utilizando paquetes binarios precompilados mediante el comando: pip install --only-binary :all: -r requirements.txt
- **Resultado:** Instalación exitosa de PyYAML en el entorno de Python sin requerir compilación local por Cython.

Problema 5: Incompatibilidad de Flask-MySQLdb

- **Problema:** Incompatibilidad de la librería Flask-MySQLdb.
- **Causa:** La versión especificada originalmente (Flask-MySQLdb==1.0.1) presentaba fallas de disponibilidad o compatibilidad con las versiones recientes de Python y controladores del sistema.
- **Solución:** Actualización de la dependencia a la versión Flask-MySQLdb==2.0.0, empleando además PyMySQL como alternativa de conexión.
- **Resultado:** Conexión estable y funcional entre la aplicación desarrollada en Flask y la base de datos MySQL/MariaDB.

Problema 6: Ejecución con Python global

- **Problema:** Ejecución del proyecto utilizando la instalación de Python global del sistema.
- **Causa:** Omisión del aislamiento del entorno de desarrollo al iniciar las pruebas del sistema, provocando conflictos entre dependencias globales del sistema operativo y las requeridas por el proyecto GroStop.
- **Solución:** Recreación total de un entorno virtual aislado mediante la ejecución de comandos dedicados.
- **Resultado:** Aislamiento exitoso de dependencias e integración limpia en el archivo requirements.txt.

9. REGISTRO DE COMANDOS UTILIZADOS

En estricto apego a las actividades de administración del sistema y control del entorno realizadas por el equipo, se ejecutaron los siguientes comandos:

1. Instalación de Git / Descarga de cliente:

```
https://git-scm.com/install/git
```

2. Clonación del repositorio:

```
git clone https://github.com/aakkadn-beep/ECommerce-Grocery-Store.git
```

3. Navegación al directorio del proyecto:

```
cd ECommerce-Grocery-Store
```

4. Instalación de requerimientos precompilados:

```
pip install --only-binary :all: -r requirements.txt
```

5. Creación del entorno virtual:

```
python -m venv venv
```

6. Activación del entorno virtual:

```
source venv/Scripts/activate
```

7. Ejecución del servidor web de Flask:

```
python run.py
```

8. Servidor en ejecución en el navegador local:

```
http://127.0.0.1:5000/
```

10. PRUEBAS DEL SISTEMA

10.1 Pruebas Realizadas

- **Prueba de arranque de servidor:** Verificación de inicialización de la aplicación ejecutando `python run.py` tras instalar `pyparsing` y solucionar `PyYAML`.
- **Prueba de persistencia de datos:** Validación de la estructura de las 13 tablas del archivo `grostop.sql` en MySQL/MariaDB y ejecución de sentencias `ON DUPLICATE KEY UPDATE`.
- **Prueba de lógica de negocio:** Validación del correcto funcionamiento del flujo de checkout, procesamiento de compras y aplicación del campo `Percentage_Discount` en cupones dentro del carrito.

10.2 Pruebas Pendientes o No Documentadas

Información no proporcionada por el equipo.

11. ARQUITECTURA Y FUNCIONAMIENTO TÉCNICO

El sistema GroStop es una aplicación web de comercio electrónico respaldada por las siguientes tecnologías:

- **Lenguaje de Programación:** Python.
- **Framework Backend:** Flask, encargado de la gestión de rutas web (`market/routes.py`), controladores y lógica de negocio.
- **Base de Datos Relacional:** MySQL / MariaDB, que almacena el modelo de datos relacional compuesto por 13 tablas estructuradas en el archivo `grostop.sql`.
- **Gestión de Entorno:** Entorno virtual nativo (`venv`) para el aislamiento estricto de librerías y generación de `requirements.txt`.
- **Control de Versiones:** Git, con un repositorio remoto centralizado en GitHub bajo la dirección <https://github.com/aakkadn-beep/ECommerce-Grocery-Store.git>.

Relación entre Aplicación y Base de Datos

La aplicación se conecta a MySQL/MariaDB a través de adaptadores de Python (`Flask-MySQLdb` / `PyMySQL`). Las peticiones enviadas desde las rutas de Flask ejecutan consultas SQL relacionales que leen y modifican el estado de los productos, usuarios, compras y cupones de descuento, manteniendo la persistencia en las 13 tablas del sistema.

12. TRABAJO COLABORATIVO Y COORDINACIÓN

La coordinación de la célula Alpha Logic Systems se basó en la división de funciones definida en el Plan de Trabajo:

- Oscar Hernández Banda actuó como Product Owner, estableciendo el puente de comunicación con el docente e identificando los flujos de usuario clave para orientar las correcciones del sistema.
- Yordi Hernández Coronado asumió el análisis arquitectónico, modelando la estructura mediante diagramas de casos de uso, clases y entidades para dar visibilidad conceptual al sistema.
- Kenia Guadalupe Hernández Torres lideró la estabilización técnica del backend en Flask, resolviendo los errores de librerías en Python y ajustando los controladores del núcleo ejecutable.
- Cristofer Yahir Herrera mantuvo la integridad del código fuente administrando la migración del repositorio en GitHub y validando el despliegue del entorno.
- Ana Fernanda Loera Mendoza auditó y corrigió la estructura de datos relacional de grostop.sql, garantizando la consistencia del tipo Percentage_Discount y las transacciones de compra.

13. RESULTADOS OBTENIDOS

1. Centralización del código: Migración y control de versiones consolidado en el repositorio GitHub oficial del proyecto.
2. Estabilización del Backend: Aplicación Flask operativa mediante el comando `python run.py`.
3. Flujos corregidos: Corrección exitosa del flujo de checkout, procesamiento de compras y validación de cupones de descuento.
4. Validación de Base de Datos: Verificación funcional del archivo grostop.sql, sus 13 tablas, el ajuste de Percentage_Discount y la eliminación de errores por registros duplicados.
5. Corrección del Entorno: Resoluciones de compatibilidad para PyYAML, pyparsing y Flask-MySQLdb==2.0.0 en Python.
6. Aislamiento de Dependencias: Entorno virtual configurado correctamente con su correspondiente archivo requirements.txt.

14. CONCLUSIÓN GRUPAL

El desarrollo del proyecto GroStop por parte de la célula Alpha Logic Systems reafirma principios fundamentales de la Ingeniería de Software:

1. Comprender código existente: La capacidad de abordar un proyecto preexistente y descifrar su arquitectura es tan vital como desarrollar sistemas desde cero.

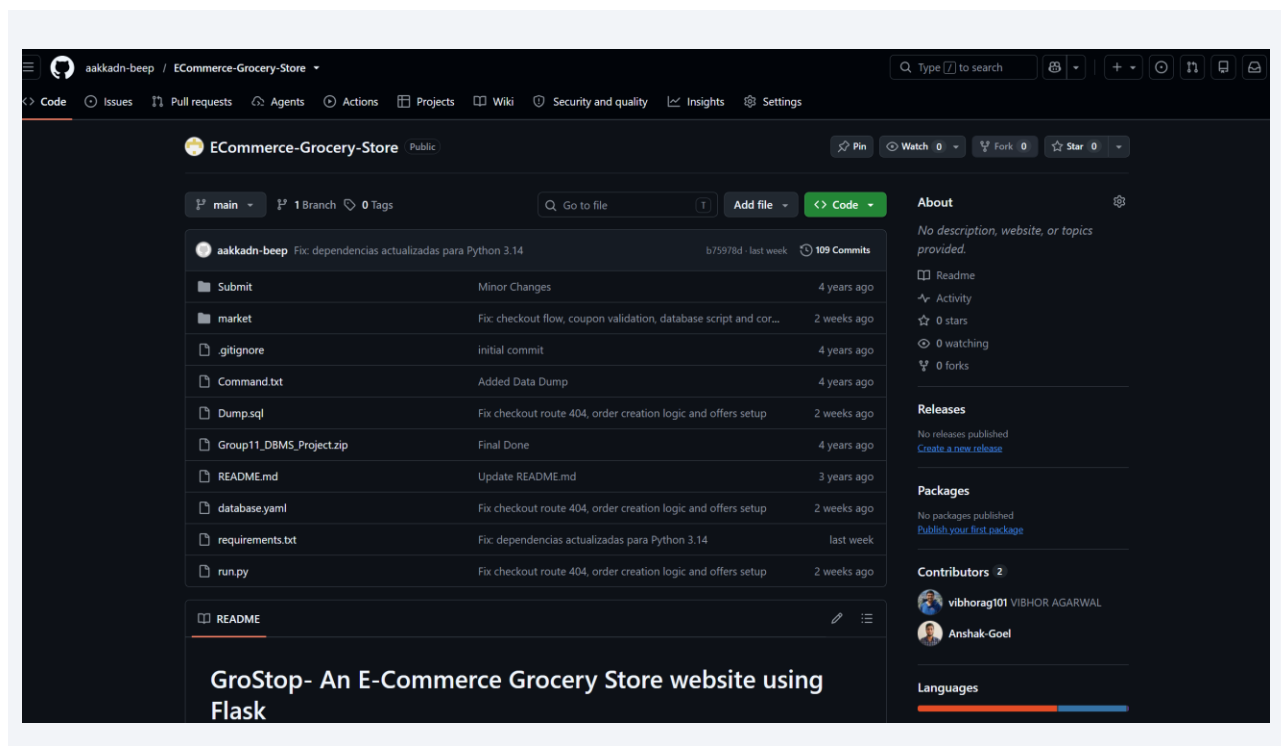
2. Control de versiones riguroso: El uso de Git y GitHub permite coordinar esfuerzos de forma ordenada sin perder trazabilidad técnica.
3. Documentación de errores: Registrar detalladamente las fallas de compatibilidad reduce los tiempos de resolución y permite documentar lecciones aprendidas.
4. Validación y Pruebas: Probar la interacción entre backend y base de datos previene incoherencias en la lógica de negocio (como el cálculo impreciso de carritos de compra).
5. Trabajo Colaborativo: La delimitación clara de roles es indispensable para el éxito de proyectos complejos de software.

15. EVIDENCIAS

Las siguientes secciones han sido maquetadas para la inclusión de las capturas de pantalla de respaldo por parte del equipo:

Evidencia 1: Repositorio de GitHub

Detalle/Referencia: <https://github.com/aakkadn-beep/ECommerce-Grocery-Store.git>



Evidencia 2: Clonación del repositorio

Detalle/Referencia: git clone https://github.com/aakkadn-beep/ECommerce-Grocery-Store.git

```
aakka@THUNDEROBOT MINGW64 ~  
$ git clone https://github.com/aakkadn-beep/ECommerce-Grocery-Store.git  
cd ECommerce-Grocery-Store|
```

Evidencia 3: Entorno virtual (directorio venv)

Detalle/Referencia: python -m venv venv

```
aakka@THUNDEROBOT MINGW64 ~/ECommerce-Grocery-Store (main)  
$ python -m venv venv  
source venv/Scripts/activate  
(venv)  
aakka@THUNDEROBOT MINGW64 ~/ECommerce-Grocery-Store (main)  
$ |
```

Evidencia 4: Base de datos (13 tablas en MySQL/MariaDB)

Detalle/Referencia: Archivo grostop.sql e interfaz phpMyAdmin/XAMPP

The screenshot shows the phpMyAdmin interface for a database named 'grostop'. The left sidebar displays the database structure, including tables like 'admin', 'admin_views', 'associated_with', 'cart', 'category', 'customer', 'delivery_boj', 'offer', 'orders', 'product', 'product_feedback', 'rates_order_delivery', 'rating_table', 'selects', 'seller', and 'sells'. The main panel shows a list of 16 tables with their respective actions (Examinar, Estructura, Buscar, Insertar, Vaciar, Eliminar) and details (Filas, Tipo, Cotejamiento, Tamaño, Residuo a depurar). The tables are listed in a table format with columns for Table, Action, Rows, Type, Collation, Size, and Residual to be purged.

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
admin	Examinar	5	InnoDB	utf8mb4_general_ci	16,0 KB	-
admin_views	Examinar	0	InnoDB	utf8mb4_general_ci	32,0 KB	-
associated_with	Examinar	10	InnoDB	utf8mb4_general_ci	48,0 KB	-
cart	Examinar	42	InnoDB	utf8mb4_general_ci	64,0 KB	-
category	Examinar	15	InnoDB	utf8mb4_general_ci	16,0 KB	-
customer	Examinar	33	InnoDB	utf8mb4_general_ci	32,0 KB	-
delivery_boj	Examinar	12	InnoDB	utf8mb4_general_ci	48,0 KB	-
offer	Examinar	5	InnoDB	utf8mb4_general_ci	32,0 KB	-
orders	Examinar	9	InnoDB	utf8mb4_general_ci	64,0 KB	-
product	Examinar	90	InnoDB	utf8mb4_general_ci	64,0 KB	-
product_feedback	Examinar	10	InnoDB	utf8mb4_general_ci	64,0 KB	-
rates_order_delivery	Examinar	0	InnoDB	utf8mb4_general_ci	64,0 KB	-
rating_table	Examinar	~0	Visualizar	--	-	-
selects	Examinar	20	InnoDB	utf8mb4_general_ci	32,0 KB	-
seller	Examinar	8	InnoDB	utf8mb4_general_ci	32,0 KB	-
sells	Examinar	28	InnoDB	utf8mb4_general_ci	32,0 KB	-

16 tablas Número de filas ~287 InnoDB utf8mb4_general_ci 640,0 KB 0 B

Evidencia 5: Ejecución del sistema

Detalle/Referencia: python run.py

```
(venv)
aakka@THUNDERBOT MINGW64 ~/ECommerce-Grocery-Store (main)
$ python run.py
* Serving Flask app 'market'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-599-548
```

Evidencia 6: Sistema funcionando en el navegador

Detalle/Referencia: <http://127.0.0.1:5000/>

